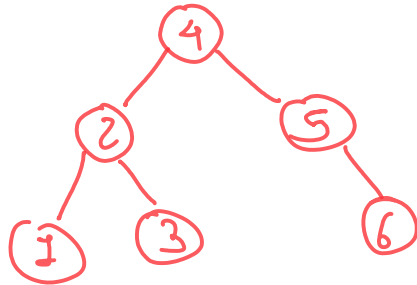


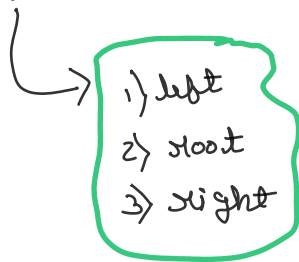
Binary Search tree (BST)

- In binary tree the search time complexity becomes $O(n)$.
- while in binary search tree the time complexity is $O(H)$
i.e equal to height of tree, we use BST for fast lookups
- BST is a binary tree only having some more additional property
 - left subtree nodes $<$ root
 - right subtree nodes $>$ root
 - left & right subtrees are also BST with no duplicates

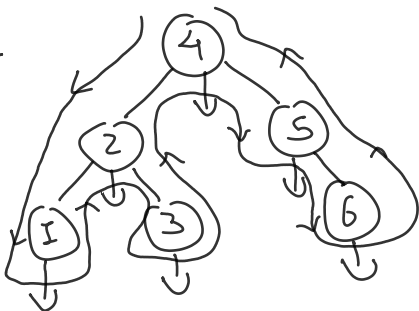


It has some special property :-

- Inorder traversal of BST gives sorted sequence



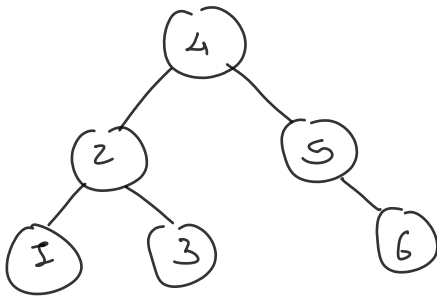
ex



Inorder $\Rightarrow 1, 2, 3, 4, 5, 6$

BST Search

ex



key = 9

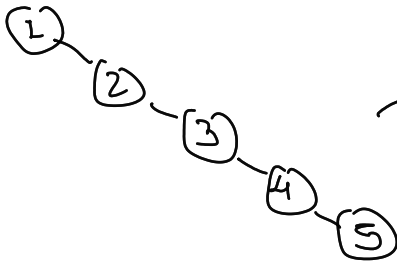
key > data → go to right

key < data → go to left

→ In balanced BST the height of the tree is $\log(n)$

→ But In worst case it becomes $O(n)$

ex skewed tree



→ here

$$O(H) = O(n)$$

Strategy

→ most problem will be solved using recursion i.e. dividing into subproblems & making recursive calls on subtrees

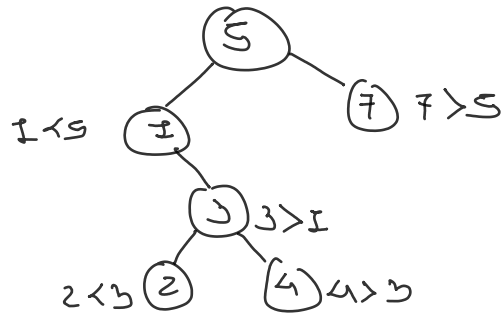
Build a BST

→ values = {5, 1, 3, 4, 2, 7}

* If the root is null Insert the node/value

* If root.data < value Insert on right

* If root.data > value Insert on left



Code

```
public static Node Insert (Node root, Int val)
{
```

```
    if (root == null)
```

```
    {
```

```
        root = new Node(val);
```

```
        return root;
```

```
    }
```

```
    if (root.data < val)
```

```
    {
```

```
        root.right = Insert (root.right, val);
```

```
    }
```

```
    if (root.data > val)
```

```
    {
```

```
        root.left = Insert (root.left, val);
```

```
    }
```

```
    return root;
```

```
}
```

```
public static void main (String args[])
```

```
{
```

```
    int values[] = {5, 1, 3, 4, 2, 7};
```

Node root = null;

for (int i=0; i < values.length; i++)

{

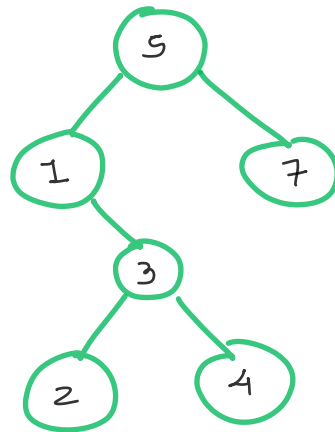
root = insert(root, values[i]);

}

// print the values using Inorder traversal

}

Search a BST



public static boolean search (Node root, int val)

{

if (root == null)

{

return false;

}

if (root.data == val)

{

return true;

}

if (root.data > val)

{

return search(root.left, val);

}

else {

return search(root.right, val);

}

}

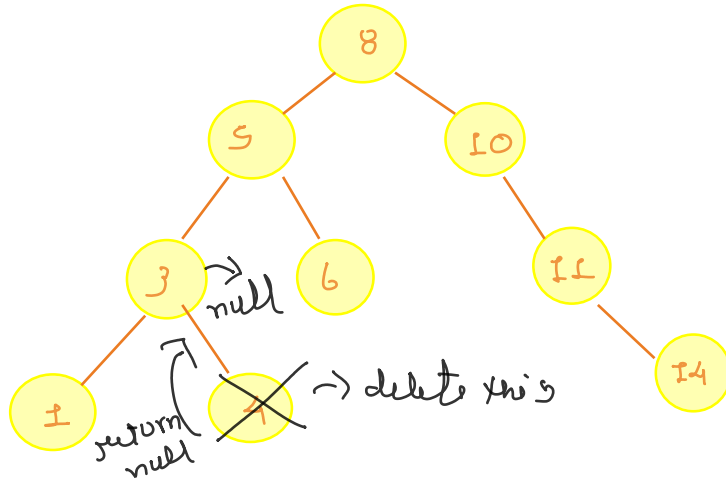
Delete node In BST

→ There are 3 cases of deleting the node in BST :-

- 1) deleting the leaf node
- 2) deleting the node with one child
- 3) deleting the node with two child

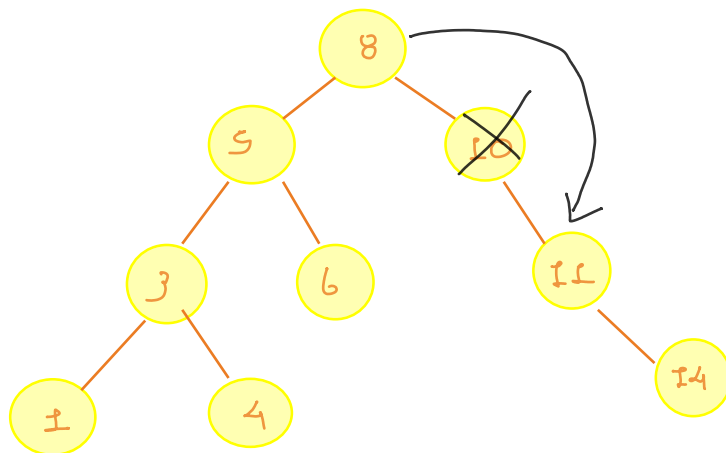
Case 1 (no child)

→ delete that node and return null to the parent

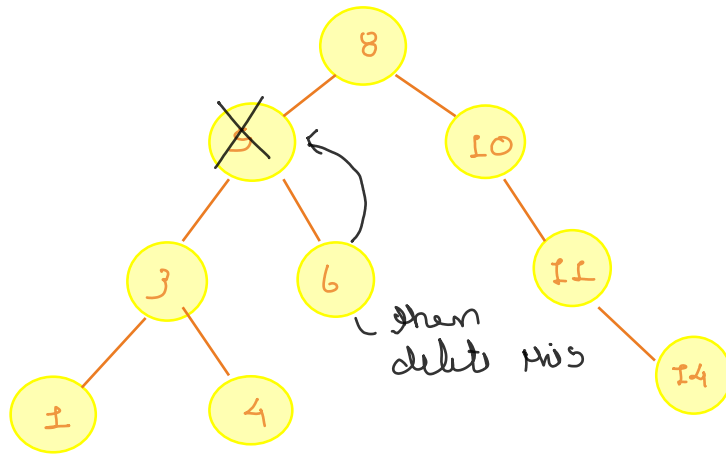


One Child

→ delete node and replace with child node



Two child



- replace value with inorder successor
- delete the node from inorder successor

ex Inorder is :- 1, 3, 4, 5, 6, 10, 11, 14

Suppose want to delete 5 then replace it with inorder successor i.e 6 and then delete 6 from its position

Code

```
public static Node delete (Node root, int val) {  
    if (root.data < val) {  
        root.right = delete (root.right, val);  
    }  
    if (root.data > val) {  
        root.left = delete (root.left, val);  
    } else {  
        if (root.left == null && root.right == null) {  
            return null;  
        } else if (root.left == null) {  
            return root.right;  
        } else if (root.right == null) {  
            return root.left;  
        }  
    }  
}
```

```

Node IS = findInorderSuccessor(root.right);
root.data = IS.data;
root.right = delete(root.right, IS.data);
}
return root;
}

```

```

public static Node findInorderSuccessor(Node root){
    while(root.left != null){
        root = root.left;
    }
    return root;
}
}

```